RSPP Library Software Complexity Report

Commit: 12273c9baf6de16aacad50f5a122510c598f0bdb

Introduction

The Software Complexity Report for the RSPPLIB is generated by Coverity. The tool supports C and C++, analyses code using its own checkers as well as MISRA checkers, and provides compexity statistics. The server responsible for continuous analysis is located at the APTIV branch in Gothenburg, Sweden and it is running Coverity Analysis version 2018.06.

Scope

This document applies to the Radar Sensor Pre-Process(TBD acronym) Roster number: F360CORE_00

The SW consists of modules which have different origin. In general each module will fall into one of the following categories:

- Third party modules directly integrated (in their original format/state) in the project.

- Third party code that are affected by our parameter configuration upon generation.

- Modules which are developed in the project by APTIV.

There are two possible approaches for generating the complexity report:

- Include all sources into the measurements including tools and third-party code which gives a more complete picture of the complexity of the whole SW. Drawback of this approach is that it shows call stacks and other information which is affected by parts that the project is unable to affect and makes it hard to detect APTIV implementation issues.

- The second approach is to only include modules which APTIV has implemented and thereby get feedback about possible issues introduced by the current project only.

Current report has been produced as the result of the first approach so that all potential risks can be identified. Issues which are not relevant will be excluded from the report during iterative review process supported by the utilized tools (Coverity).

Analyzed builds

Following builds are analyzed by Coverity:

- **RSPPLib** which is a library to be used when building Radar Tracker applications for radar sensors used with F360 systems.

Additionally there is a **Combined view** which provides results for all the above builds combined.

Lines of Code

Lines of Code metric is calculated as sum of lines in all files reported by Coverity as being part of a given build. If given line is compiled independently during builds for different cores it is counted each time in given build perspective, but in **Combined view** it is counted only once. That's why LOC in **Combined view** is not simply a sum of LOC from other builds.

build name	lines of code	comment lines	blank lines	total lines	files
Combined view	18797	11139	4758	34694	173
F360CORE_00_RSPF	18797	11139	4758	34694	173

Big files

File size is calculated as a sum of the following metrics reported by Coverity:
- lines of code
- empty (blank) lines
- comment lines

Summary

Table below presents how many files has more lines than the given thresholds.

build name	>10000	>5000	>2000	>1000
Combined view	0	1	2	8
F360CORE_00_RSPP_LIB	0	1	2	8

Details

Tables below presents files with high number of lines. Files longer than 5000 lines are marked RED.

Combined view

file name	lines of code	blank lines	comment lines	total
/usr/include/c++/9/bits/stl_algo.h	3583	559	1743	5885
/usr/include/x86_64-linux-gnu/c++/9/bits/c++config.h	913	507	663	2083
/usr/include/c++/9/bits/stl_algobase.h	919	151	392	1462
/usr/include/math.h	1053	132	156	1341
/usr/include/c++/9/bits/stl_iterator.h	766	205	328	1299
/source/rspp_sensor_capability.cpp	383	67	627	1077
/usr/include/stdlib.h	619	184	222	1025
/source/rspp_math_func.cpp	335	59	614	1008
/usr/include/c++/9/bits/algorithmfwd.h	489	183	182	854
/source/rspp_math_func.h	382	55	230	667

F360CORE_00_RSPP_LIB

file name	lines of code	blank lines	comment lines	total
/usr/include/c++/9/bits/stl_algo.h	3583	559	1743	5885
/usr/include/x86_64-linux-gnu/c++/9/bits/c++config.h	913	507	663	2083
/usr/include/c++/9/bits/stl_algobase.h	919	151	392	1462
/usr/include/math.h	1053	132	156	1341
/usr/include/c++/9/bits/stl_iterator.h	766	205	328	1299
/source/rspp_sensor_capability.cpp	383	67	627	1077
/usr/include/stdlib.h	619	184	222	1025
/source/rspp_math_func.cpp	335	59	614	1008
/usr/include/c++/9/bits/algorithmfwd.h	489	183	182	854
/source/rspp_math_func.h	382	55	230	667

High comment ratio

Comment ratio is calculated out of the following metrics reported by Coverity:
- lines of code
- empty (blank) lines
- comment lines

Summary

Table below presents how many files that have a higher comment ratio than the given thresholds.

build name	>95%	>90%	>85%	>80%
Combined view	0	0	0	2
F360CORE_00_RSPP_LIB	0	0	0	2

Details

Tables below presents files with high comment ratio. Files with ratio higher than 80% are marked RED.

Combined view

file name	lines of code	blank lines	comment lines	comment ratio
/usr/include/x86_64-linux-gnu/bits/long-double.h	1	4	16	76.19%
/source/rspp_sort_data_type.h	20	4	69	74.19%
/source/rspp_update_sensors_time.cpp	12	3	40	72.73%
/source/rspp_norm_heading_angle.cpp	17	3	48	70.59%
/source/rspp_filter_detections_with_bad_azimuth_confidence.cpp	16	3	41	68.33%
/source/rspp_compute_detections_properties.cpp	51	11	128	67.37%
/usr/include/x86_64-linux-gnu/c++/9/bits/cpu_defines.h	3	8	22	66.67%
/source/rspp_set_default_flags_raw_detections.cpp	18	2	38	65.52%
/usr/include/x86_64-linux-gnu/bits/timesize.h	5	4	16	64.00%
/source/rspp_update_sensor_first_detection_index.cpp	22	3	44	63.77%

F360CORE_00_RSPP_LIB

file name	lines of code	blank lines	comment lines	comment ratio
/usr/include/x86_64-linux-gnu/bits/long-double.h	1	4	16	76.19%
/source/rspp_sort_data_type.h	20	4	69	74.19%
/source/rspp_update_sensors_time.cpp	12	3	40	72.73%
/source/rspp_norm_heading_angle.cpp	17	3	48	70.59%
/source/rspp_filter_detections_with_bad_azimuth_confidence.cpp	16	3	41	68.33%
/source/rspp_compute_detections_properties.cpp	51	11	128	67.37%
/usr/include/x86_64-linux-gnu/c++/9/bits/cpu_defines.h	3	8	22	66.67%
/source/rspp_set_default_flags_raw_detections.cpp	18	2	38	65.52%
/usr/include/x86_64-linux-gnu/bits/timesize.h	5	4	16	64.00%
/source/rspp_update_sensor_first_detection_index.cpp	22	3	44	63.77%

Low comment ratio

Comment ratio is calculated out of the following metrics reported by Coverity:
- lines of code
- empty (blank) lines
- comment lines

Summary

Table below presents how many files that have a lower comment ratio than the given thresholds.

build name	<5%	<10%	<15%	<20%
Combined view	2	5	17	28
F360CORE_00_RSPP_LIB	2	5	17	28

Details

Tables below presents files with low comment ratio. Files with ratio lower than 20% are marked RED.

Combined view

file name	lines of code	blank lines	comment lines	comment ratio
/usr/lib/gcc/x86_64-linux-gnu/9/include/stdint.h	14	0	0	0.00%
/usr/include/x86_64-linux-gnu/bits/types/__sigset_t.h	8	2	0	0.00%
/usr/include/c++/9/bits/predefined_ops.h	272	68	22	6.08%
/source/rspp_sensor_capability.h	113	18	11	7.75%
/usr/include/x86_64-linux-gnu/bits/endianness.h	7	3	1	9.09%
/usr/include/x86_64-linux-gnu/sys/types.h	166	42	24	10.34%
/usr/include/x86_64-linux-gnu/bits/types/struct_timespec.h	22	3	3	10.71%
/usr/include/x86_64-linux-gnu/bits/types/timer_t.h	5	3	1	11.11%
/usr/include/x86_64-linux-gnu/bits/types/time_t.h	5	3	1	11.11%
/usr/include/x86_64-linux-gnu/bits/types/sigset_t.h	5	3	1	11.11%

F360CORE_00_RSPP_LIB

file name	lines of code	blank lines	comment lines	comment ratio
/usr/lib/gcc/x86_64-linux-gnu/9/include/stdint.h	14	0	0	0.00%
/usr/include/x86_64-linux-gnu/bits/types/__sigset_t.h	8	2	0	0.00%
/usr/include/c++/9/bits/predefined_ops.h	272	68	22	6.08%
/source/rspp_sensor_capability.h	113	18	11	7.75%
/usr/include/x86_64-linux-gnu/bits/endianness.h	7	3	1	9.09%
/usr/include/x86_64-linux-gnu/sys/types.h	166	42	24	10.34%
/usr/include/x86_64-linux-gnu/bits/types/struct_timespec.h	22	3	3	10.71%
/usr/include/x86_64-linux-gnu/bits/types/timer_t.h	5	3	1	11.11%
/usr/include/x86_64-linux-gnu/bits/types/time_t.h	5	3	1	11.11%
/usr/include/x86_64-linux-gnu/bits/types/sigset_t.h	5	3	1	11.11%

Cyclomatic Complexity (McCabe)

Cyclomatic complexity is a quantitative measure of the number of linearly independent paths through a program's source code. More details about this metric can be found at
https://en.wikipedia.org/wiki/Cyclomatic_complexity

Summary

Table below presents how many functions that have a higher cyclomatic complexity than the given thresholds.

build name	>120	>60	>30	>15
Combined view	0	0	0	1
F360CORE_00_RSPP_LIB	0	0	0	1

Details

Tables below presents functions with high cyclomatic complexity. Functions with complexity greater than 15 are marked RED.

Combined view

Function name	File name	lines	blocks	cyclomatic complexity
rspp_variant_A::Input_Sanity_Check(rspp_variant_A::RSPP_Detection_List_Tag &, rspp_variant_A::F360_Radar_Sensor_Tag (&)[10], const rspp_variant_A::RSPP_Calibrations_Tag &)	rspp_input_sanity_check.cpp	50	37	17

F360CORE_00_RSPP_LIB

Function name	File name	lines	blocks	cyclomatic complexity
rspp_variant_A::Input_Sanity_Check(rspp_variant_A::RSPP_Detection_List_Tag &, rspp_variant_A::F360_Radar_Sensor_Tag (&)[10], const rspp_variant_A::RSPP_Calibrations_Tag &)	rspp_input_sanity_check.cpp	50	37	17

Halstead Effort

Halstead Effort is a software metric that is based on number of operands and operators. More details about this metric can be found at https://en.wikipedia.org/wiki/Halstead_complexity_measures

Summary

Table below presents how many functions that have a higher Halstead effort than the given thresholds.

build name	>5000000	>500000	>50000	>5000
Combined view	0	0	0	1
F360CORE_00_RSPP_LIB	0	0	0	1

Details

Tables below presents functions with high Halstead effort. Functions with Halshead effort greater than 5000 are marked RED.

Combined view

Function name	File name	operands	operations	Halstead effort
rspp_variant_A::Input_Sanity_Check(rspp_variant_A::RSPP_Detection_List_Tag &, rspp_variant_A::F360_Radar_Sensor_Tag (&)[10], const rspp_variant_A::RSPP_Calibrations_Tag &)	rspp_input_sanity_check.cpp	22	11	7175.79

F360CORE_00_RSPP_LIB

Function name	File name	operands	operations	Halstead effort
rspp_variant_A::Input_Sanity_Check(rspp_variant_A::RSPP_Detection_List_Tag &, rspp_variant_A::F360_Radar_Sensor_Tag (&)[10], const rspp_variant_A::RSPP_Calibrations_Tag &)	rspp_input_sanity_check.cpp	22	11	7175.79